Zhuo Chen
COMP 562
Proj #1

1) **Question 1:**
For numerically calculating the Jacobian in getjacobianmatrix(), please try with different values of the finite difference (i.e., different values of Δq where q denotes the joint angles). Please report how different values of the finite difference affect the accuracy of the Jacobian-based control. Please also report what is a good value for this finite difference Δq.

Solution:

Δq	Position Error (m)	Orientation Error (rad)
1e-2	0.000674	0.007744
1e-3	0.000322	0.007759
1e-5	0.000339	0.007976
1e-6	0.000496	0.005011

The data shows that the choice of finite difference step size Δq affects the Jacobian-based control accuracy. When $\Delta q = 10^{-2}$, the position error is the largest, which suggests that the step size is too large to approximate the Jacobian accurately. When Δq is reduced to 10^{-3} and 10^{-5}, the position error becomes smaller and stays relatively stable, which indicates better numerical approximation. For $\Delta q = 10^{-6}$, the orientation error becomes the smallest, but the position error increases again slightly, which may be caused by numerical precision issues when the step size is too small.

Overall, the results suggest that a moderate finite difference value works better than a very large or very small one. Based on this table, $\Delta q = 10^{-3}$ is a good choice because it gives the smallest position error and also keeps the orientation error at a reasonable level.

2) **Question 2:**
Report: Please read the codes of PandaSimėxecute() in simpy (the one you finished in Task 1) and understand how these two checkers (i.e., is state valid() and is state high quality()) are used. Please briefly report how the two checkers will affect the execution of the robot, respectively.

Solution:
The two checkers are used during the execution of Cartesian velocity controls in PandaSim.execute().

The is_state_high_quality() checker evaluates the manipulability of the robot using the Jacobian matrix. This check is performed at every simulation step. If the manipulability is below the threshold, the configuration is considered close to a singularity, and the execution of the control is terminated. This prevents the robot from entering configurations where the Jacobian-based control becomes unstable or inaccurate.

The is_state_valid() checker verifies whether the current state of the robot is physically valid. This check is performed periodically during execution (every 0.2 seconds). It ensures that the robot remains within the workspace bounds, satisfies the state space bounds, and does not collide with the ground. If an invalid state is detected, the execution is stopped and the motion is rejected.

Together, these two checkers ensure that the robot motion is both physically valid and kinematically reliable during the execution of planned controls.

3) **Question 3:**
Please show your method and the relevant findings in the report. To show the result, you are encouraged to add figures of the initial and optimized trajectories to your report for comparison

Method:

For Task 4, I chose to optimize Task 2. My method is to first generate an initial Task 2 plan using the kinodynamic RRT planner, and then optimize this plan with a shortcut-based algorithm.

The reason for doing this is that the initial RRT plan is not efficient(see fig 1 and 2). It may contain many unnecessary intermediate nodes, which increases both the path length and the total execution cost. To improve the plan, I wrote a shortcut function that tries to replace a long segment of the original path with a more direct connection.

The basic idea is the following:

Starting from the current node in the plan, I try to connect it directly to a later node in the path. This is done by calling the control sampler to generate a control toward the target state. If the sampled control can reach a state that is close enough to that later node, then I accept this shortcut and skip all intermediate nodes between the two nodes. Otherwise, I keep the next node from the original plan and continue the same process. In my implementation, I always try farther nodes first, so that I can remove as many unnecessary nodes as possible when a shortcut succeeds.

I measure the quality of a plan using its total cost, which is defined as the sum of the durations of all controls in the plan. Let Δt_i be the duration of the control used at step i. Then the total plan cost is

$$J(\pi) = \sum_{i=1}^{N-1} \Delta t_i.$$

In the code, this corresponds to summing the fourth element of each control vector, namely `ctrl[3]`.

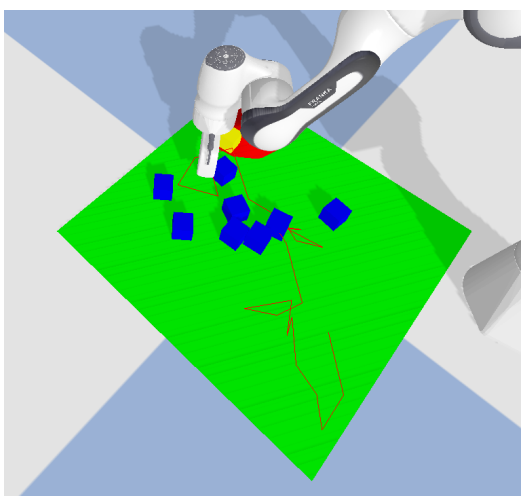

Figure 1: Task 2 example

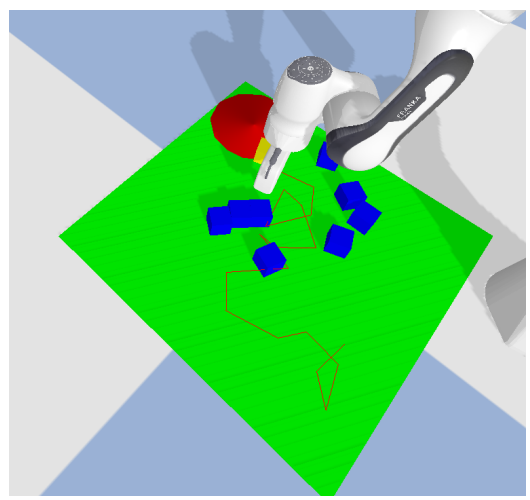

Figure 2: Task 2 example

Findings:

The function `try_shortcut()` in opt.py checks whether the planner can replace a long part of the original path with a more direct connection. It samples controls from `from_node` toward `to_node`, and accepts the shortcut only if the sampled motion reaches close enough to the target node.

Here, `num_trials=30` means the sampler tries 30 candidate controls for one shortcut attempt. `dist_threshold=0.12` means the shortcut is accepted only when the reached state is within distance 0.12 of the target state.

Findings:

After my experiments, I found that the shortcut optimization can reduce both the number of nodes and the total plan cost. The red lines indicates the initial task 2 plan. The blue line indicates the optimized plan. And the change is in purple circle.

In fig 2 , the initial plan had 18 nodes and cost 8.8173, while the optimized plan had 16 nodes and cost 7.7087. This means the shortcut method removed 2 nodes and reduced the cost by about 1.11. In Example 2, the initial plan had 10 nodes and cost 4.4390, while the optimized plan had 9 nodes and cost 3.8825. In this case, the method removed 1 node and reduced the cost by about 0.56.

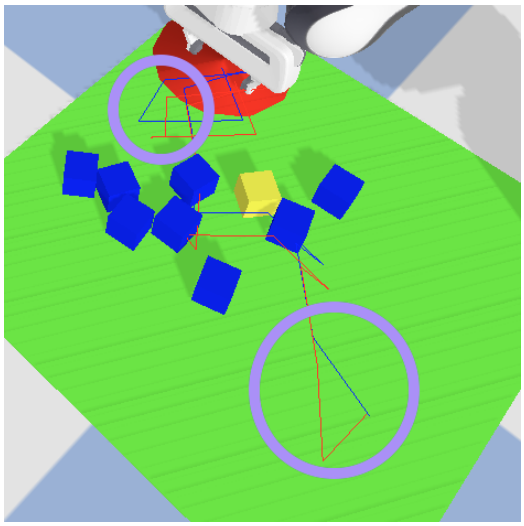

Figure 3: Task 4 example 1

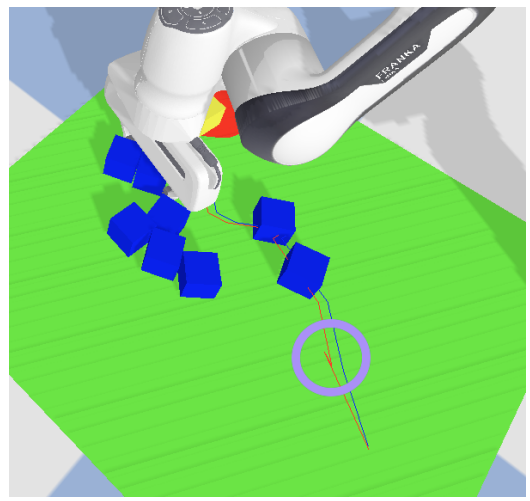

Figure 4: Task 4 example 2